2 sorted arrays.

1) i < n && j < m
2) i < n
3) j < m

3 sorted arrays

1) i < n1 && j < ~~n~~ n2 && k < n3] 3
2) j < n2 && k < n3
3) i < n1 && k < n3] 2
4) i < n1 && j < n2
5) i < n1
6) j < n2] 1
7) k < n3

Instead of the above, we will do:

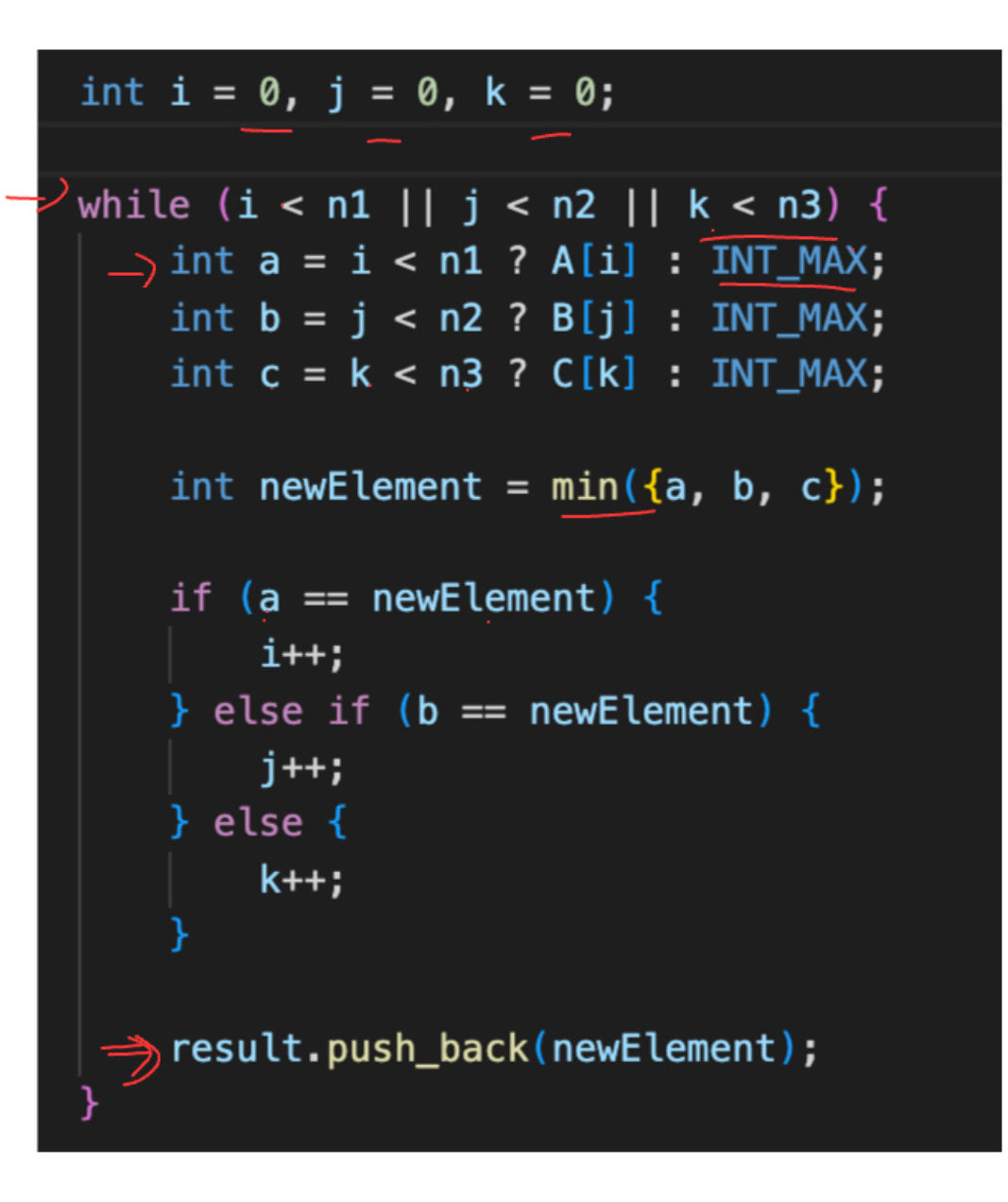
1) i < n1 || j < n2 || k < n3

$0 \le A, B, C \le 2^{31} - 1$

A = [1, 2, 3]

B = [1, 1, 4]

C = [5, 6, 7, 8]



```
int i = 0, j = 0, k = 0;

while (i < n1 || j < n2 || k < n3) {
    int a = i < n1 ? A[i] : INT_MAX;
    int b = j < n2 ? B[j] : INT_MAX;
    int c = k < n3 ? C[k] : INT_MAX;

    int newElement = min({a, b, c});

    if (a == newElement) {
        i++;
    } else if (b == newElement) {
        j++;
    } else {
        k++;
    }

    result.push_back(newElement);
}
```

a = ~~1 2 3~~ ∞

b = ~~1 4~~ ∞

c = ~~5 6 7~~ 8

nE = ~~1 2 3 4 5 6 7~~ 8

res:
[1, 1, 1, 2, 3, 4, 5, 6, 7, 8]

TC:

---

arr: [900, 940, 950, 1100, 1500, 1800], ~~1500~~

dep: [910, 1200, 1120, 1130, 1900, 2000], ~~1800~~

1 1 2 3 1 2

arr: [900, 940, 950, 1100, 1500, 1800]

res: dep: [910, 1200, 1120, 1130, 1900, 2000]

- 1 → ~~1~~ 900
- 2 → 1120
- 3 → 1130

Brute Force:

TC: $O(n^2) + n \log n) = O(n^2)$

AS: $O(n)$.

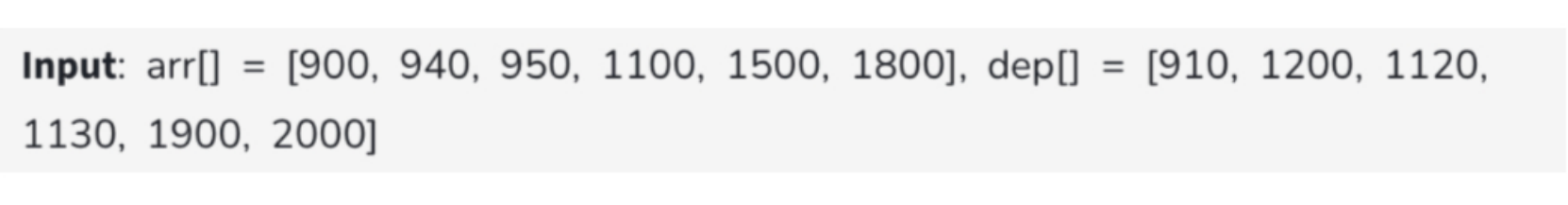
900
1100
1

800
1200
2

1250
~~1000~~
1300
1

HINT: ~~No~~ Platforms need = max no. of trains co-existing on the station at the same time.

x - y



Input: arr[] = [900, 940, 950, 1100, 1500, 1800], dep[] = [910, 1200, 1120, 1130, 1900, 2000]